CSD361 — Lab Assignment 3

Unsupervised Learning, Probability, Gaussian Mixture Models, Principal Component Analysis & Neural Networks

Topics: K-Means clustering • Probability, MLE & MAP • Gaussian Mixture Models & EM • PCA / covariance / eigenvalues / SVD • Neural Networks & Backpropagation

Deadline: 12 November 2026, 11:59 PM.

Objectives

- Apply K-Means clustering and reason about its distortion objective, the elbow method, initialization, and limitations.
- Estimate distribution parameters with Maximum Likelihood (MLE) and Maximum A Posteriori (MAP), and relate priors to regularization.
- Fit a Gaussian Mixture Model and understand Expectation-Maximization through responsibilities and parameter updates.
- Perform PCA via the covariance matrix and via SVD, analyze eigenvalues, positive semidefiniteness, and explained variance.
- Build and train a neural network: forward pass, activation functions, backpropagation, and training behaviour.
- Compare generative and discriminative approaches and justify a model choice from evidence.

Dataset and software

- **Dataset:** Wine dataset — from sklearn.datasets import load_wine (178 samples, 13 features, 3 classes). Use this one dataset throughout. Where a 2-D visualization helps, use two of its features (do not switch datasets).
- **Software:** Python 3 with numpy, pandas, matplotlib, scikit-learn, scipy. The assignment must run in Google Colab.
- **Reproducibility:** set random_state=42 (and np.random.seed(42)) wherever randomness is involved. Standardize features where appropriate, and fit any scaler on the **training** data only for the supervised parts.

Instructions

- **Do all numerical work in Python.** Compute every distance, estimate, EM update, eigenvalue/SVD, and gradient in code, and then **interpret** the output.
- For each question, show your code, the actual executed output, any required plot (with title, axis labels, and legend), and a short written interpretation.
- Demonstrate mathematical understanding through derivations in words, explanations of the equations you implement, interpretation of results, and comparisons — not by running a library function and only reporting its number.
- Do not hard-code or fabricate outputs. Label every figure and state what the reader should observe.

Submission Guidelines:

1. Read each question carefully and provide a clear, brief, and well-structured solution.
2. For programming or implementation-based questions :
    a. Write readable and well-commented code, and an output screenshot is required.
    b. Clearly mention the dataset, preprocessing steps, algorithms, and parameters used.
    c. Include relevant output, graphs, or performance measures where required.
3. Prepare the documentation with all code screenshots and output screenshots (wherever applicable) and upload a solution PDF.
4. Also, provide the link to your code file in the document itself (for example, Jupyter Notebook, lab, or Colab), whatever you are using. We will be using the same code file during the viva for this assignment.
5. Use Python 3 with NumPy, pandas, Matplotlib, and scikit-learn. Set the random seed = 42 everywhere.
6. A single executed Google Colab notebook (.ipynb) with all code, outputs, and figures, organized by section.
7. A detailed report containing your interpretations, derivations, and comparisons.

8. Ensure the notebook runs top-to-bottom in a clean Colab runtime with random_state=42.
9. The deadline for the submission is 12 November 2026, 11:59 PM.
10. Please note that this assignment contains 50 marks, and 50 marks will be for the related VIVA. Total -100 marks.
11. The viva will be conducted on Friday, 13 November 2026, for Assignment 3 during Lab hours.

Section A — K-Means Clustering & Distance-Based Reasoning [10 marks]

K-Means partitions data into K clusters by minimizing the distortion $J(c,\mu) = \Sigma_i \|x^{(i)} - \mu_{c(i)}\|^2$. *Use the standardized Wine features.*

A.1 Explain why feature scaling matters for a distance-based method. Using code, compute the distance from one Wine sample to each of the three class means **before and after** standardization, and show that the nearest mean (and hence the cluster it would join) can change. Quantify how much a single large-scale feature contributes to the raw distance. ***[3 marks]***

A.2 Run K-Means for K = 1…..8 and record the distortion J (inertia). Plot J vs K and identify the elbow. Fit K-Means with K = 3 and compare the clusters to the true labels. Interpret briefly in 2-3 lines what the elbow tells you. ***[4 marks]***

A.3 Explain why K-Means is guaranteed to converge, and why only to a local minimum. Support your explanation by printing the distortion across iterations of a manual run and confirming it is non-increasing. ***[3 marks]***

Section B — Probability, Maximum Likelihood & MAP [10 marks]

B.1 From the labels, estimate the class priors P(y). For one feature, fit a Gaussian likelihood $p(x|y)$ per class and, for a chosen value x_0, use Bayes' rule to obtain the posterior $P(y|x_0)$. Explain the difference between a likelihood and a posterior (why one need not sum to 1 across classes and the other must). ***[3 marks]***

B.2 With a Gaussian prior $\mu \sim N(\mu_0, \sigma_0^2)$ and known data variance, the MAP estimate of the mean is a precision-weighted average of the data mean and the prior mean. Implement it. Compute MLE and MAP for the **full** class and for a **small subsample**, and show that the prior dominates when data are scarce while MAP $\rightarrow$ MLE as N grows. Plot the two estimates against N. ***[4 marks]***

B.3 Explain the relationship between MAP estimation and regularization. Which prior corresponds to L2 (ridge) and which to L1 (lasso)? Under what condition does MAP reduce to MLE? ***[3 marks]***

Section C — Gaussian Mixture Models & Expectation-Maximization [10 marks]

C.1 Explain why a GMM introduces a latent variable for cluster membership, how the mixing coefficients act as prior probabilities, and write the responsibility (posterior) formula $\gamma_k(x)$ and how it follows from Bayes' rule. ***[2 marks]***

C.2 Using two Wine features, fit a 3-component GMM and inspect the responsibilities (soft assignments). Visualize them and confirm, in code, that each point's responsibilities sum to 1. ***[2 marks]***

C.3 Implement **one full EM iteration by manual code**: the E-step (compute responsibilities from current parameters) and the M-step (update π_k, μ_k, Σ_k). Print the parameters before and after, confirm the responsibilities sum to 1 and the effective counts N_k sum to N, and show what will happen to log-likelihood (whether it increases or decreases). ***[3 marks]***

C.4 Compare GMM with K-Means (soft vs hard assignment, covariance shape). Support the comparison with code (e.g. where the two agree and where the GMM is uncertain) and state when you would prefer each. ***[3 marks]***

Section D — PCA, Covariance, Eigenvalues & SVD [8 marks]

D.1 Center the data and construct the covariance matrix $\Sigma = (1/m)\, X^{cT} X^{c}$ in code. Verify it is symmetric and positive semidefinite (all eigenvalues ≥ 0). Explain mathematically **why** a covariance matrix is symmetric and PSD, and why PSD (not necessarily positive definite). ***[3 marks]***

D.2 Eigendecompose Σ. Verify the reconstruction $\Sigma = V \Lambda V^{T}$. Interpret the eigenvalues (variance) and eigenvectors (directions), compute the explained-variance ratios, draw a scree plot, and determine how many components are needed for $\geq 95\%$ of the variance. ***[3 marks]***

D.3 Project the data to 2-D and visualize the classes. Contrast PCA on **raw** vs **standardized** features and explain how an unscaled high-variance feature can dominate the first principal component. ***[2 marks]***

Section E — Neural Networks & Backpropagation [12 marks]

E.1 Show numerically that stacking two linear layers (no activation) is equivalent to a single linear layer, and that inserting a ReLU breaks this. Explain why a non-linearity is necessary. ***[2 marks]***

E.2 Build the forward pass of a small MLP (e.g. 13 $\rightarrow$ 8 with ReLU $\rightarrow$ 3 with softmax) on one standardized sample. Print the hidden activations and output probabilities, and compute the number of trainable parameters for your architecture. ***[3 marks]***

E.3 Train an MLP on the standardized Wine training set (scaler fit on train only). Report the test accuracy, plot the training-loss curve, and show the effect of varying the hidden-layer size (underfitting vs adequate capacity). ***[3 marks]***

E.4 Take the computational graph $f(x,y,z) = (x+y)\cdot z$ at $x=-2$, $y=5$, $z=-4$. Compute the gradients $\partial f/\partial x$, $\partial f/\partial y$, $\partial f/\partial z$ by the chain rule **in code**, and verify them against a numerical (finite-difference) gradient. Explain the upstream/local/ downstream gradient pattern at the add and multiply gates. ***[4 marks]***

**********END**********